API INSPECTOR

GUIDE: Dr. JYOTISH K JOHN

Famitha M T FIT22CSD021
Nandana S FIT22CSD048
Neha Amina Riyaz FIT22CSD049
Ziya K V FIT22CSD066

April 5, 2025



Introduction

- API Unit Tester is a specialized software testing tool that ensures API endpoints function correctly.
- It allows developers to automate the validation of API requests and responses.
- Provides detailed reporting, debugging support, and error simulation for better testing.



Problem Statement

APIs play a critical role in enabling seamless communication between applications and services. However, developers often face challenges in testing, debugging, and validating APIs efficiently. Traditional manual testing methods can be time-consuming and error-prone, leading to inconsistencies in API behavior across different environments. The API Inspector App aims to address these challenges by providing an intuitive, automated, and user-friendly platform for API validation.



Purpose And Need

- Automates API testing, debugging, and validation, reducing manual effort.
- Identifies and fixes API issues early, improving software reliability.
- Enhances development workflow by integrating with CI/CD pipelines.
- Provides detailed reports and analytics for better API performance monitoring.
- Supports multiple API protocols, making it versatile for various applications.



Objectives

The API Inspector App aims to streamline API testing and debugging by:

- To provide an intuitive UI for sending requests, analyzing responses, and supporting authentication (e.g., JWT).
- To enable automated API validation, response formatting, and real-time debugging with logging.
- To enhance performance monitoring and ensuring compliance with expected standards.



Scope

- **Boundaries:**
  - API Inspector focuses on automating API testing, debugging, and validation.
  - Supports RESTful APIs.
  - Limited to functional, integration, and performance testing.
- **Deliverables:**
  - A web-based application for API testing and validation.
  - API request and response monitoring with reports.
  - Integration with CI/CD pipelines for automated testing.
  - User-friendly UI for managing and executing API tests.

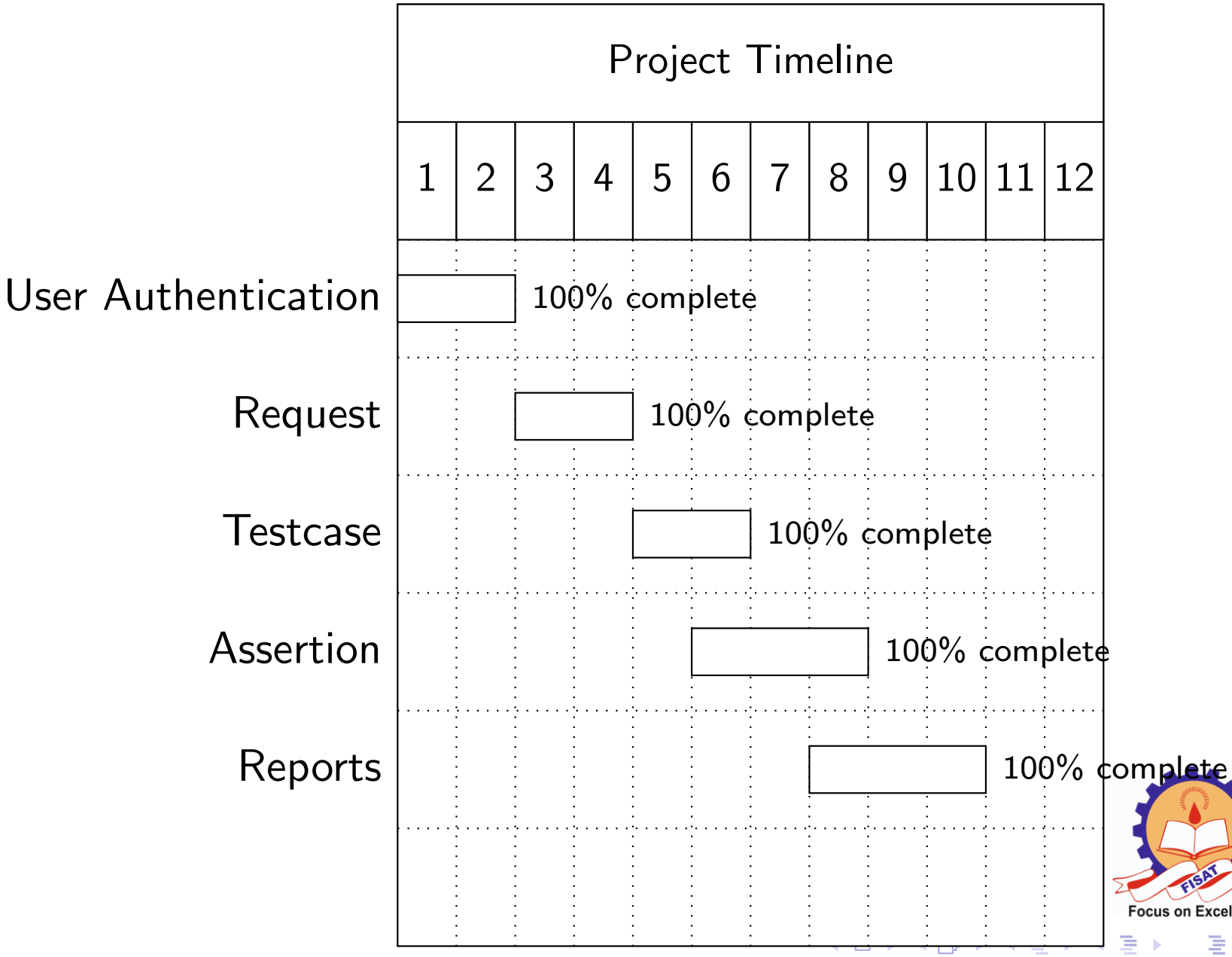


Social Relevance

- **Improving API Reliability:** Ensures that web services and applications function correctly, reducing downtime and improving user experience.
- **Enhancing Cybersecurity:** Helps identify vulnerabilities in APIs, preventing data breaches and ensuring secure communication between systems.
- **Supporting Software Development:** Assists developers and testers in debugging and validating APIs, leading to higher-quality software products.
- **Encouraging Automation:** Reduces manual testing efforts by automating API validation, increasing efficiency and accuracy in software development.
- **Empowering Open-Source and Research:** Contributes to API standardization and best practices, benefiting both industry professionals and academic researchers.



Project Scheduling



Software Requirement Specification (SRS)

Functional Requirements

- **Request Handling:** The system must allow users to send API requests using various HTTP methods (GET, POST, PUT, DELETE, etc.), including support for headers, query parameters, and request bodies.
- **Authentication:** The system should support authentication mechanisms such as API keys , JWT, and Basic Authentication to access protected APIs.
- **Test Execution:** Users must be able to define and execute test cases for APIs, including validation of responses, status codes, and execution of assertions.
- **Error Simulation:** The system should provide the capability to simulate different failure scenarios, such as timeouts, incorrect payloads, unauthorized access, and server errors.
- **Reporting:** The system must generate reports containing execution results.



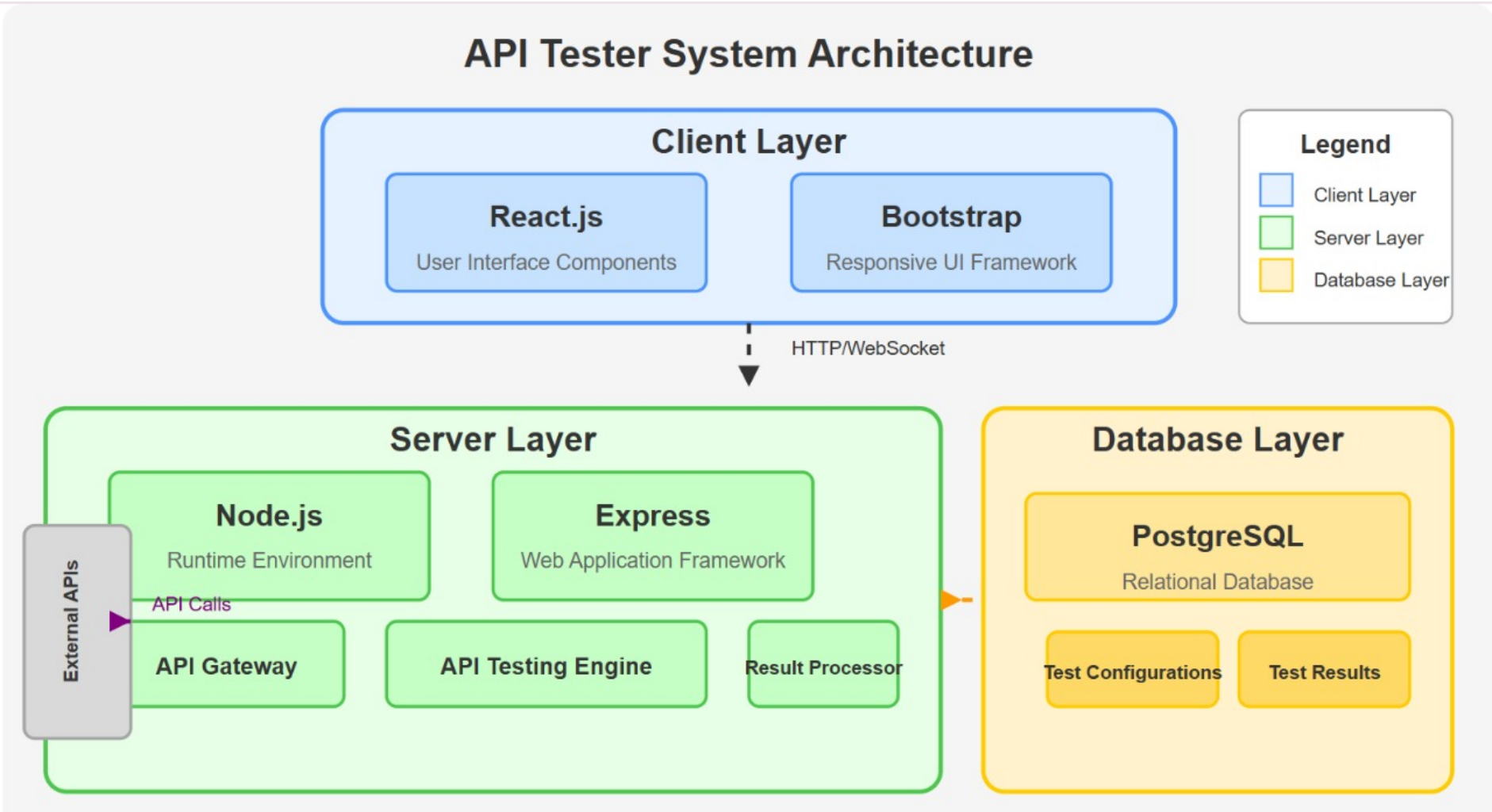
Software Requirement Specification (SRS)

Non-Functional Requirements

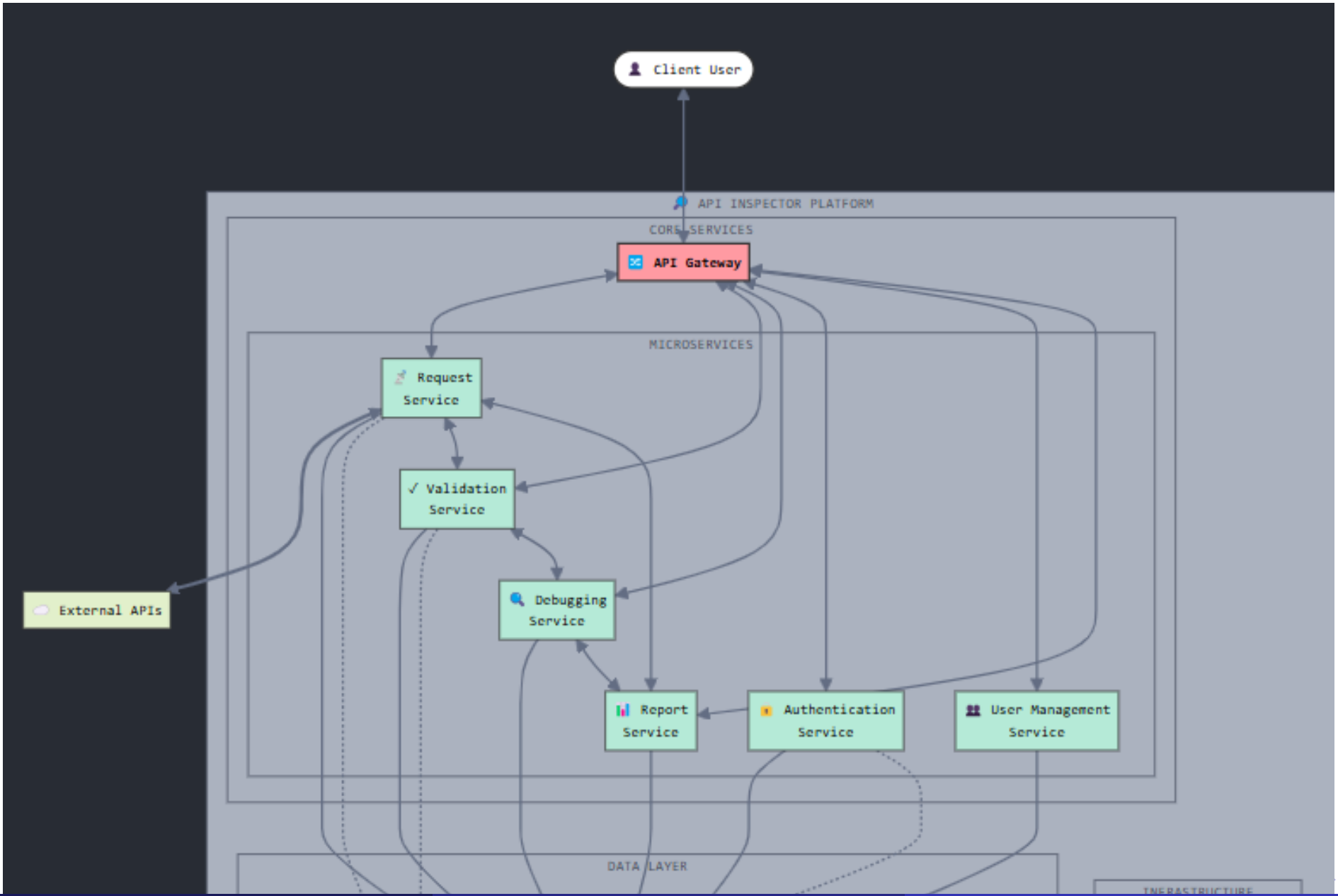
- **Performance:** The system should provide fast API request execution and handle multiple requests efficiently.
- **Scalability:** The architecture must support increased workloads and concurrent API testing without performance degradation.
- **Reliability:** The system should be highly available, minimizing downtime and ensuring accurate test execution results.



System Architecture



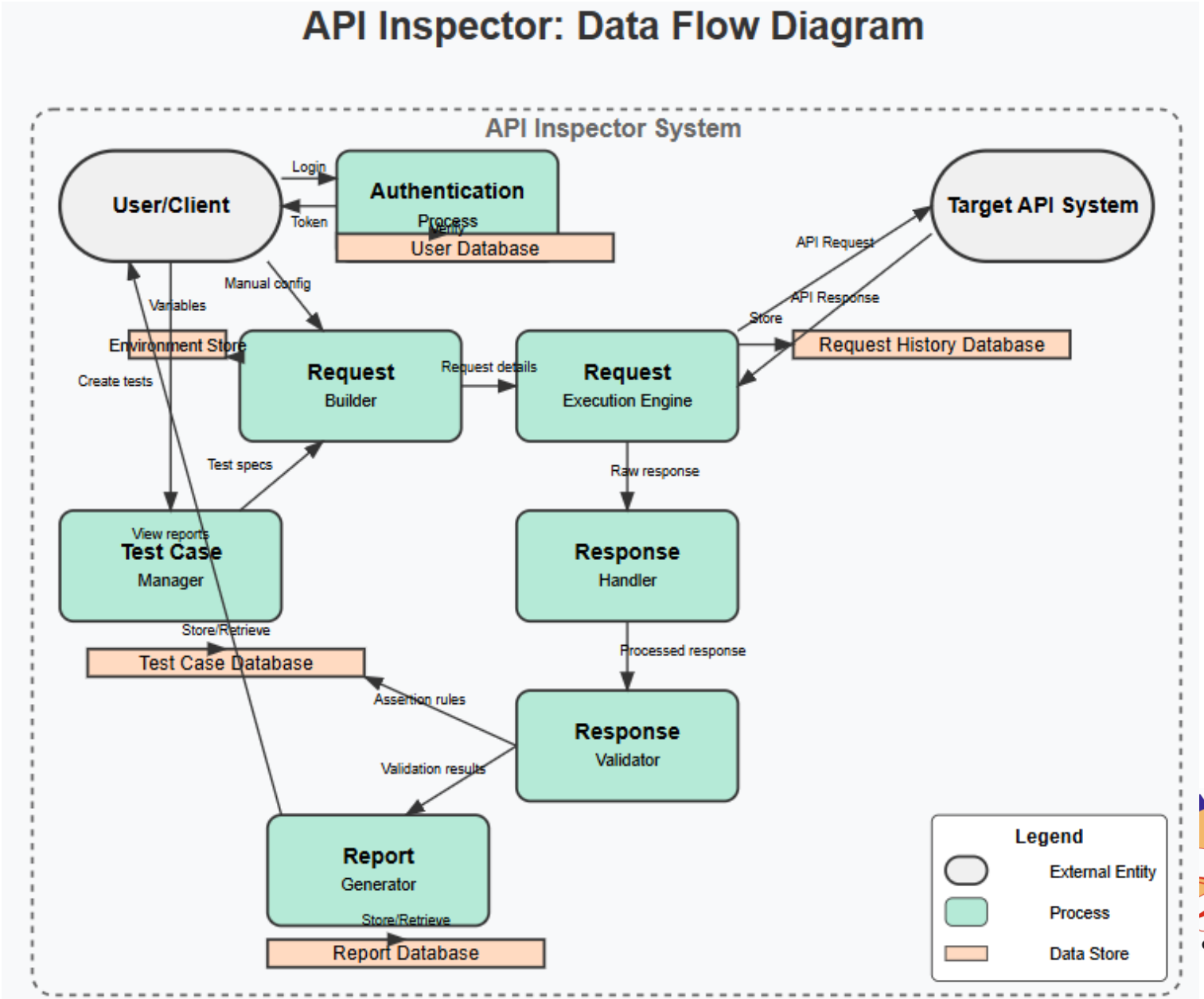
Application Architecture



Module Diagram



Data Flow Diagram



Technology Stack

- **Frontend:** React.js, Bootstrap
- **Backend:** Node.js, Express.js
- **Authentication:** JWT
- **Database:** PostgreSQL/MySQL (SQL)
- **API Communication:** REST
- **Testing Frameworks:** Jest(backend),Selenium(frontend)
- **Containerization & Orchestration:** Docker
- **CI/CD:** Gitlab
- **Version Control:** GitLab
- **Cloud Services:** AWS (EC2,RDS)



GUI Design (Mockups)

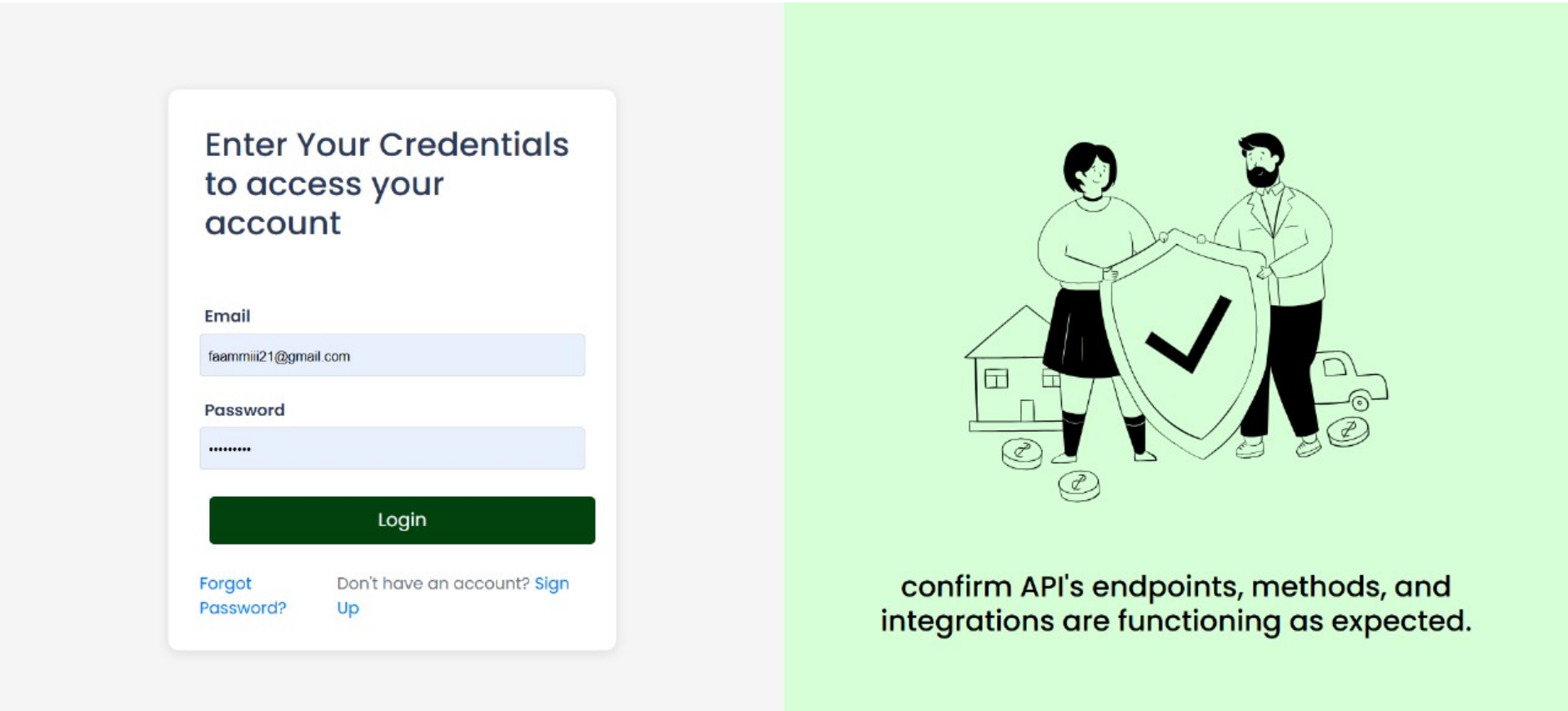


Figure: User Authentication Screen

GUI Design (Mockups)

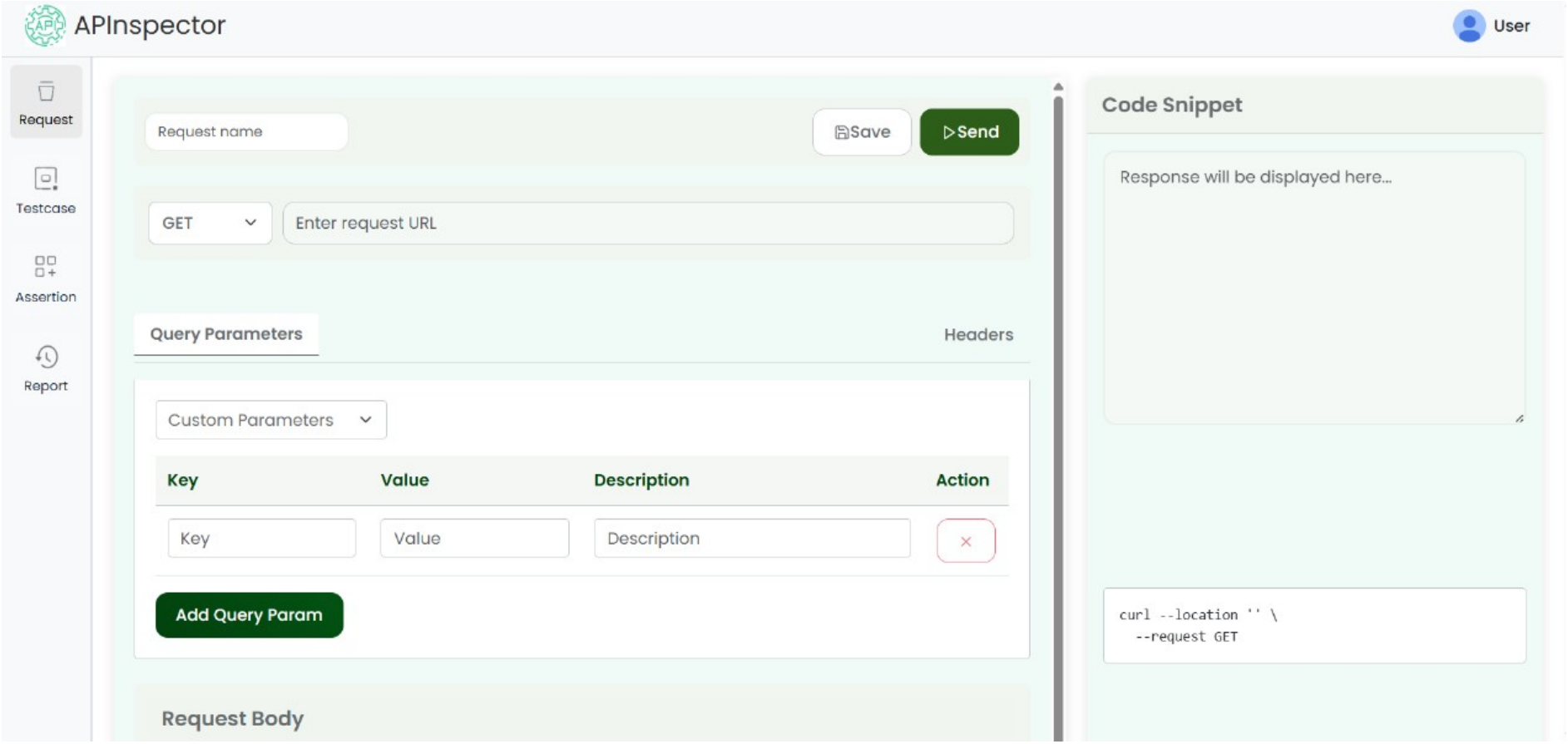


Figure: Request Screen

GUI Design (Mockups)

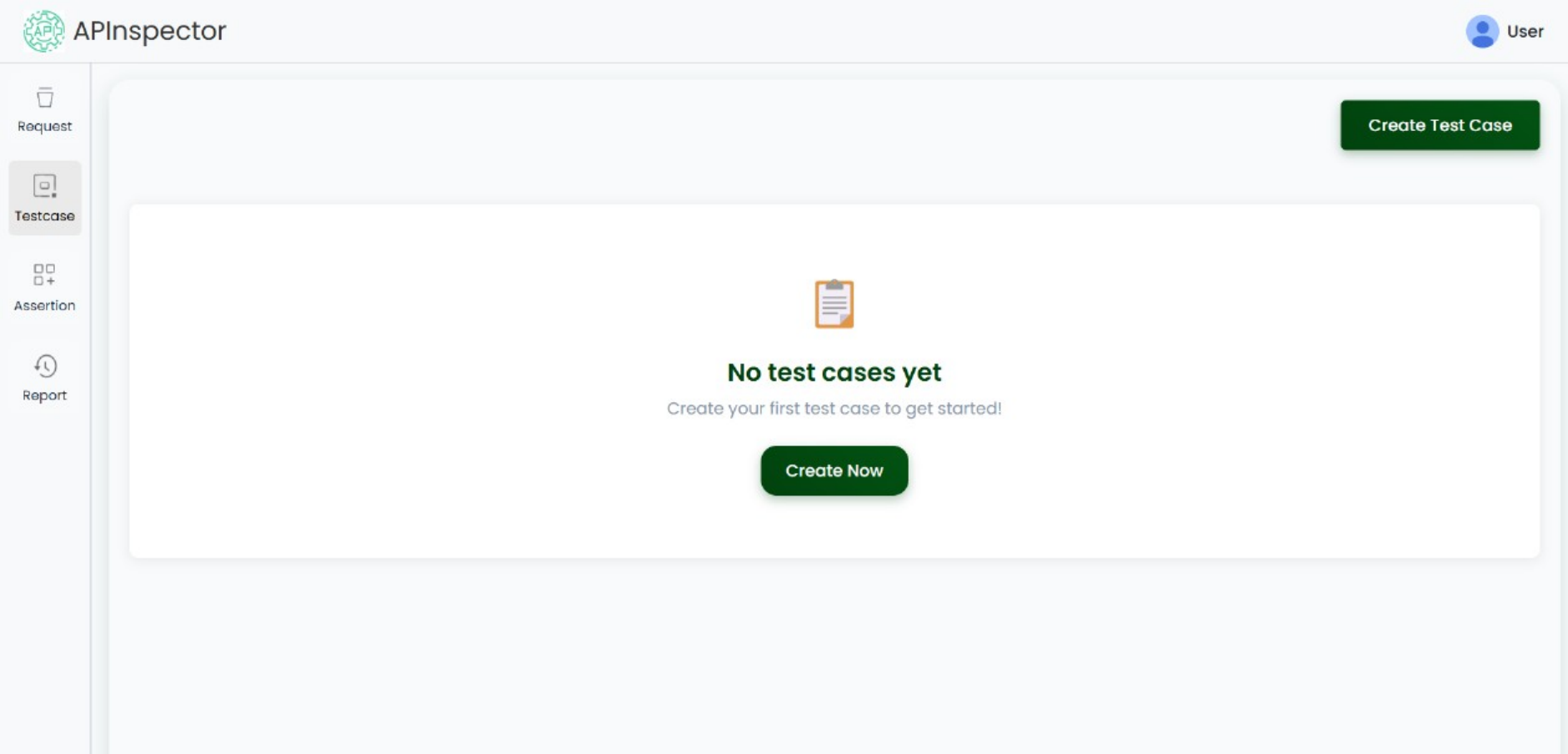


Figure: Testcase Screen

GUI Design (Mockups)

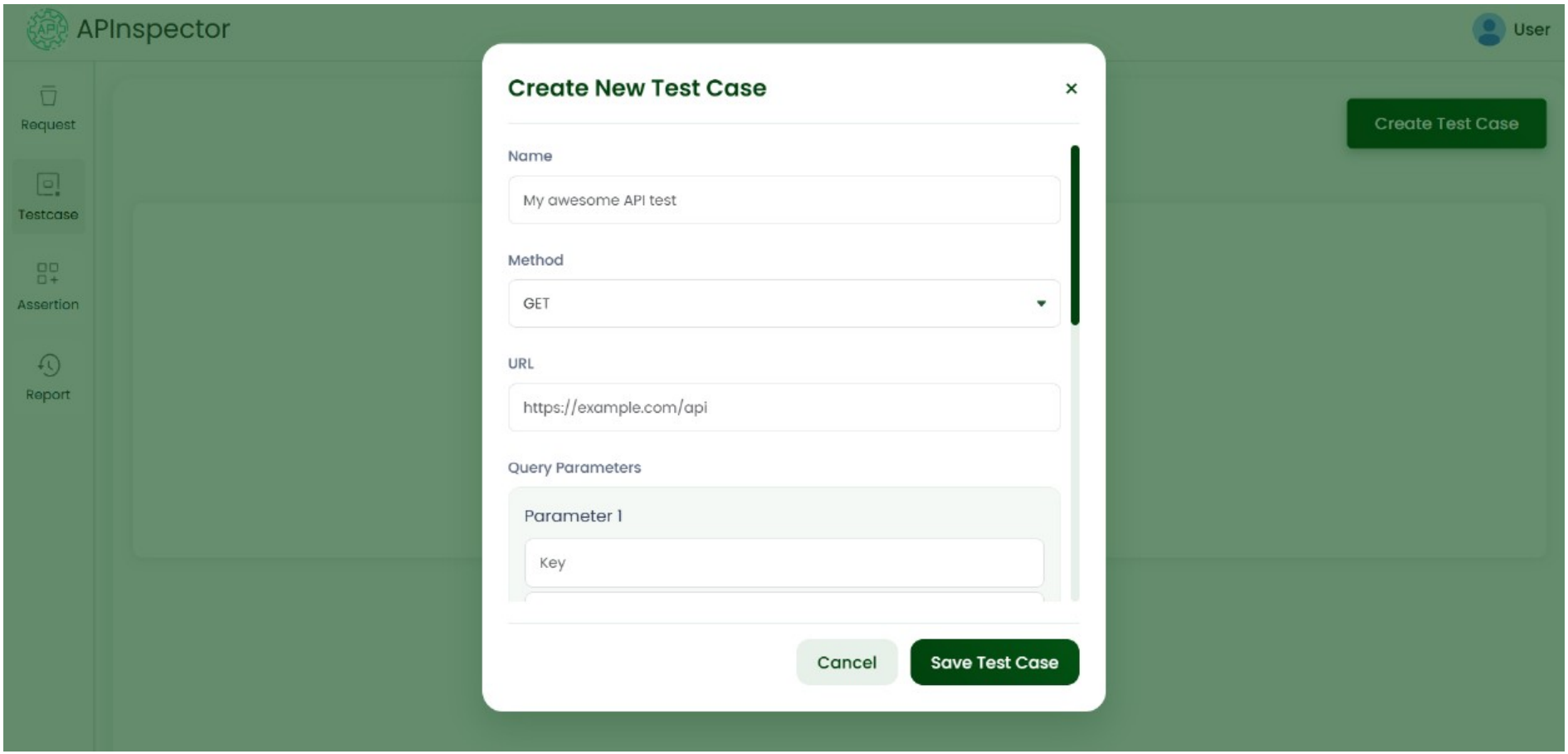


Figure: Testcase2 Screen

GUI Design (Mockups)

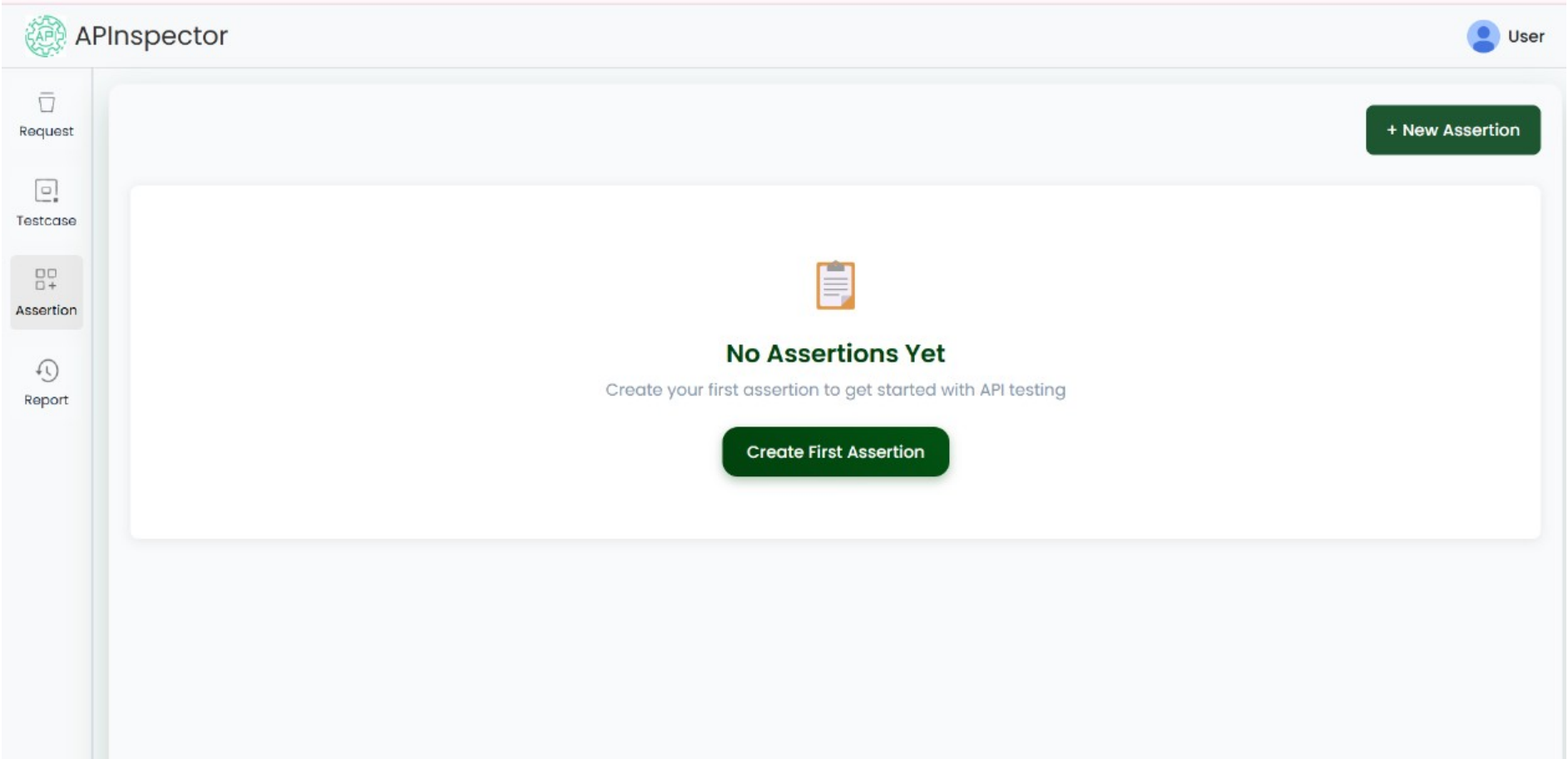


Figure: Assertion Screen

GUI Design (Mockups)

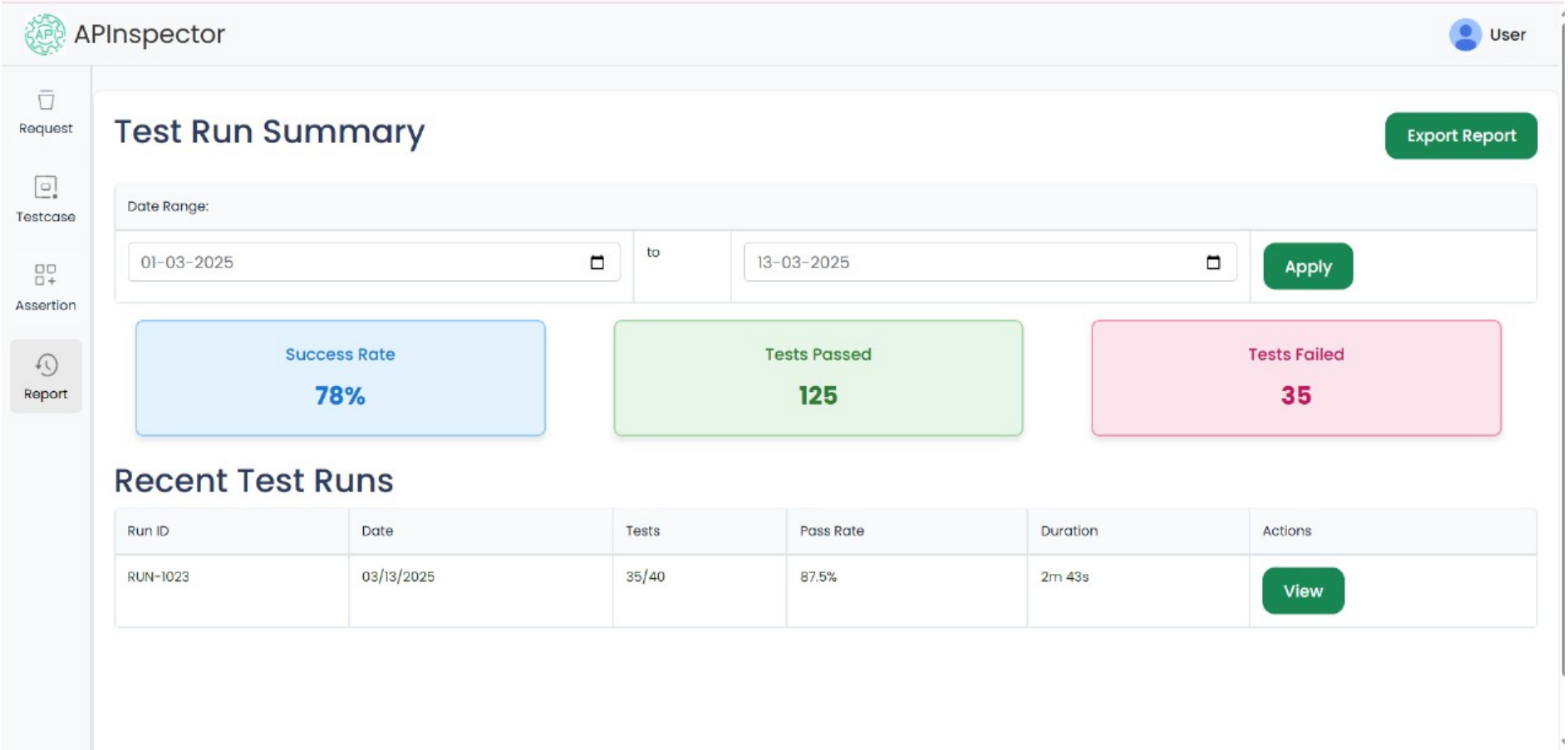
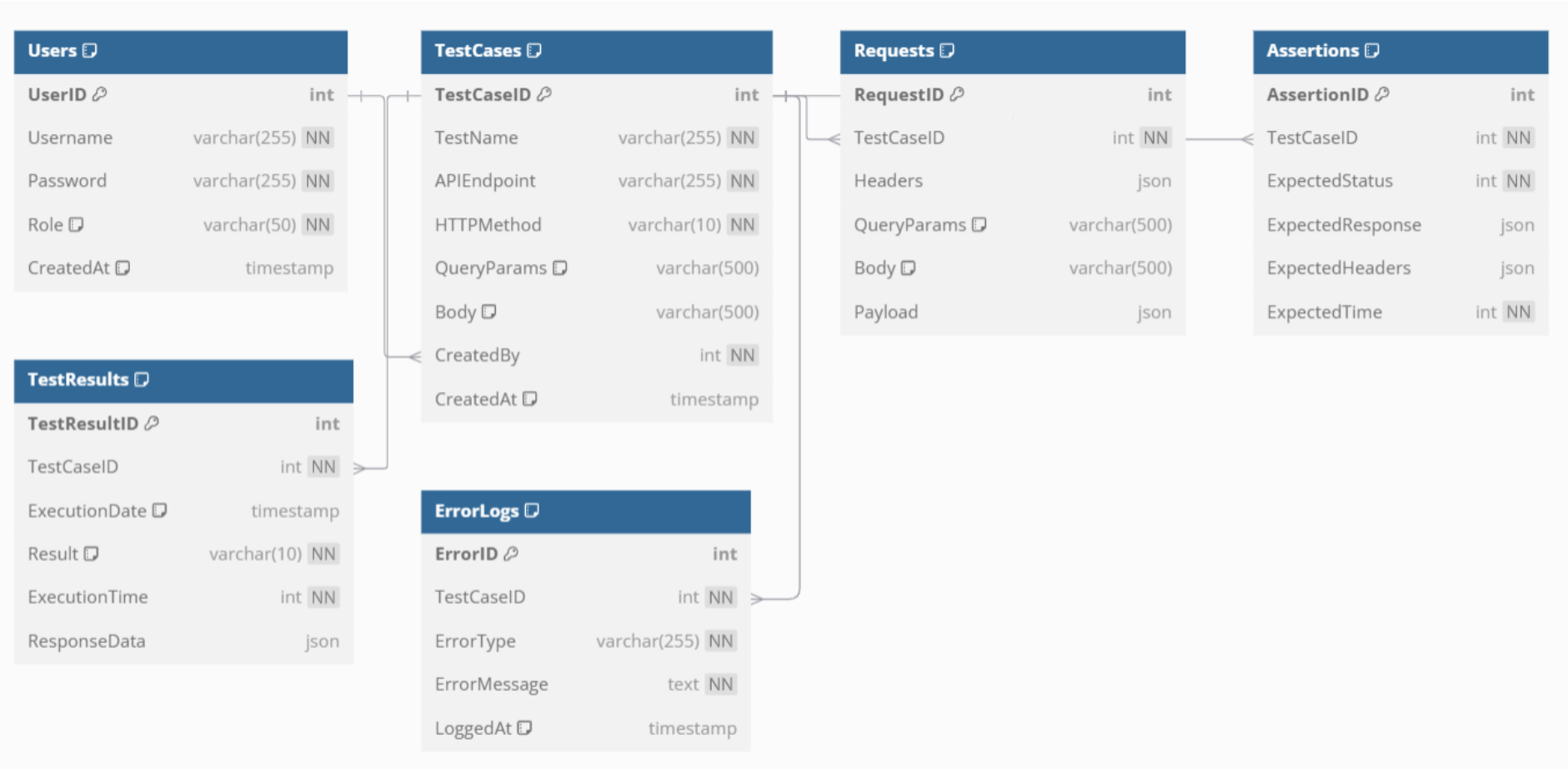


Figure: Reports Screen

Database Design



Focus on Excellence

API Endpoints

- **POST /api/auth/login** - User login, generates OTP.
- **POST /api/verify-otp** - Verifies OTP, returns JWT.
- **POST /api/forgot-password** - Initiates password reset.
- **POST /api/reset-password** - Resets password.
- **POST /api/request/send** - Sends API request.
- **POST /api/test/create** - Creates a test case.



Project Modules Timeline

Scheduled Completion Timeline

Module	Scheduled Completion Date	Completed Date
User Module	27/12/2024	25/01/2025
Request Configuration	10/03/2025	18/03/2025
Test Case Management	12/02/2025	19/03/2025
Assertion Module	16/03/2025	26/03/2023
Reports Module	20/03/2025	01/04/2025



User Module-User Authentication

The User Authentication functionality allows users to log in securely:

- The user enters their credentials (email and password).
- The system validates the credentials against the database.
- If valid, the user is granted access to the application.
- If invalid, an error message is displayed prompting the user to try again.

Testing:

- **Login:** A valid user with correct credentials should be successfully logged in.
- **Error Handling:** An error message should be displayed for invalid credentials.



Sign-Up Process

The Sign-Up process allows users to create a new account:

- The user enters their email, username, and password on the sign-up page.
- The system validates the email format and password strength.
- If the email and password are valid, the system creates a new account.
- A confirmation email is sent to the user's email address to verify the email address.

Testing:

- **Valid Input:** Ensure that the account is created successfully when the user provides valid data.
- **Email Verification:** The system should send a confirmation email to the provided email address.
- **Error Handling:** The system should display appropriate error messages for invalid email format or weak password.



Forgot Password

The Forgot Password functionality allows users to reset their password securely:

- The user requests a password reset by providing their registered email address.
- The system sends a password reset link to the registered email.
- The user follows the link and sets a new password to regain access.

Testing:

- **Password Reset Link:** The user should receive a password reset link via email.
- **Password Reset:** The user should be able to reset the password successfully by following the link.



Request Module

The Request module allows users to configure and send API requests:

- The user provides the following input:
  - **URL:** The API endpoint to which the request is sent.
  - **Method:** The HTTP method (GET, POST, PUT, DELETE) for the request.
  - **Body:** The request body data (for methods like POST, PUT).
  - **Headers:** Custom headers for the request (e.g., Authorization).
- Upon clicking the **Send** button, the request is sent.



Request Module (Continued)

Request Handling with RabbitMQ:

- **RabbitMQ** is used for queuing the request for processing asynchronously.
- The system processes the queued request and generates the API response.
- The API response is displayed in the system, showing the result of the API call.

Testing:

- **Valid Request:** Ensure that the system sends the request with the correct method, URL, headers, and body data.
- **Request Queuing:** Verify that RabbitMQ queues the request and processes it asynchronously.
- **Response Handling:** Ensure that the system displays the correct response for a valid API call.
- **Error Handling:** The system should handle errors (e.g., invalid URL, missing headers) and display appropriate error messages.



Test Case Module - Managing Test Cases

The Test Case Module allows users to create, edit, and execute test cases for API validation:

- Users can define test cases with expected inputs and outputs.
- Each test case is associated with a specific API request.
- The system executes test cases and compares responses with expected results.
- Test case results are stored for review and reporting.

Testing:

- **Test Execution:** The system should execute test cases and return the correct pass/fail status.
- **Validation:** Mismatches between expected and actual responses should be flagged as failures.
- **Editing:** Users should be able to modify and delete test cases.



Assertion Module - Validating API Responses

The Assertion Module ensures that API responses meet expected conditions:

- Users define assertions to validate specific response parameters.
- Assertions check response status codes, headers, and body content.
- The system executes assertions automatically when test cases run.
- Failed assertions indicate deviations from expected API behavior.

Testing:

- **Condition Matching:** Assertions should correctly validate API responses.
- **Error Detection:** If an assertion fails, the system should flag the error.
- **Custom Rules:** Users should be able to define and apply custom assertion rules.



Report Module - Test Execution Reports

The Report Module provides detailed insights into test execution results:

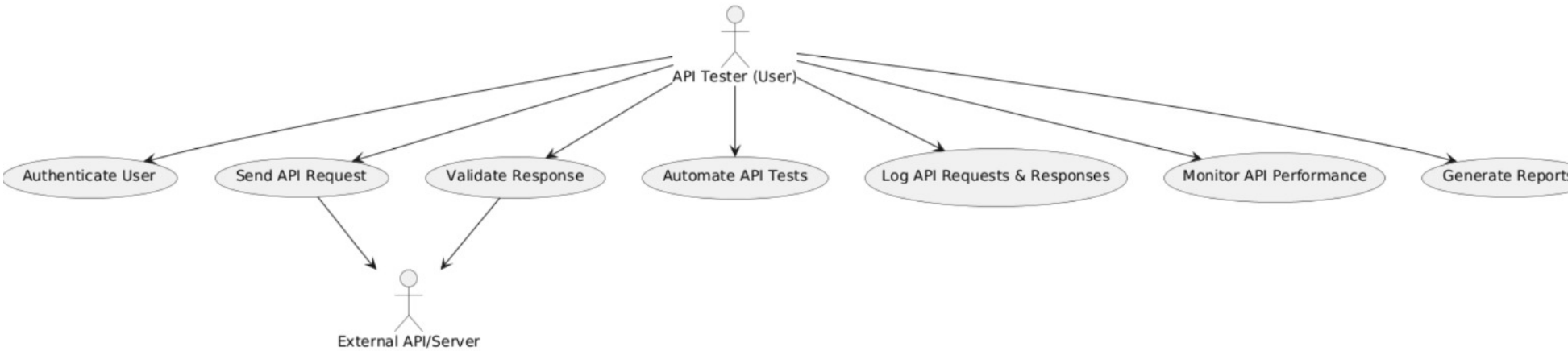
- Captures test case execution results, including passed and failed cases.
- Displays assertion failures and discrepancies in API responses.

Testing:

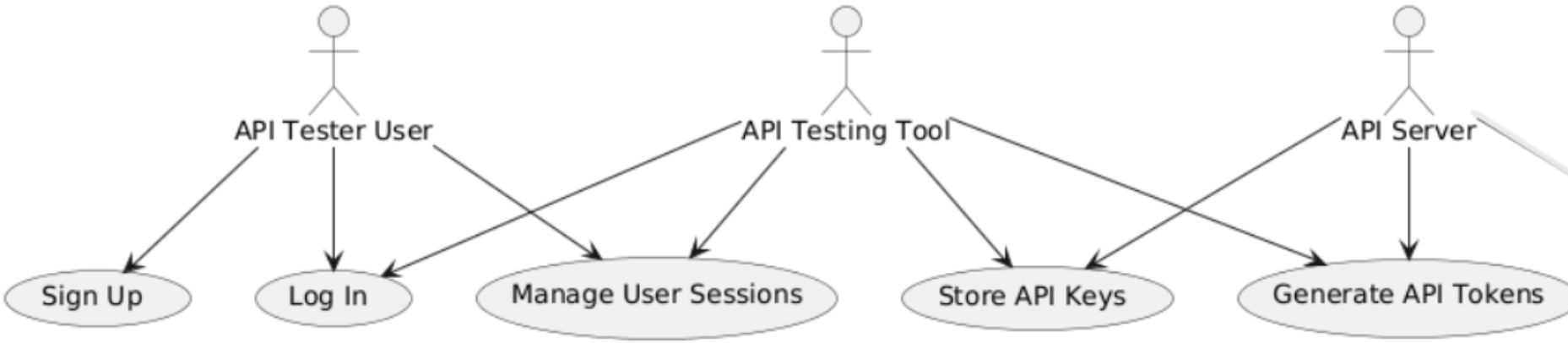
- **Accuracy:** Reports should correctly reflect test case results.
- **Error Logging:** Assertion failures and unexpected responses should be clearly logged.
- **Export Functionality:** Users should be able to generate and download reports in multiple formats.



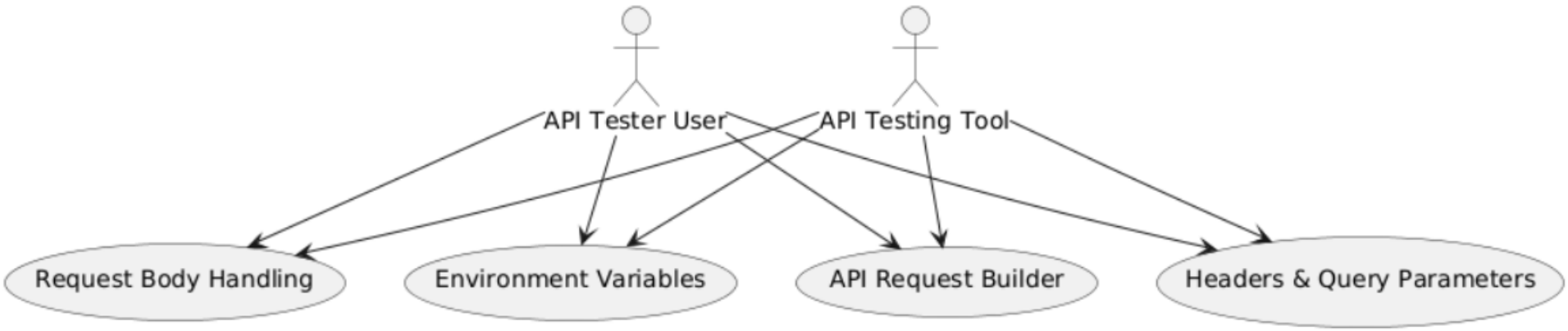
Use Case Diagram



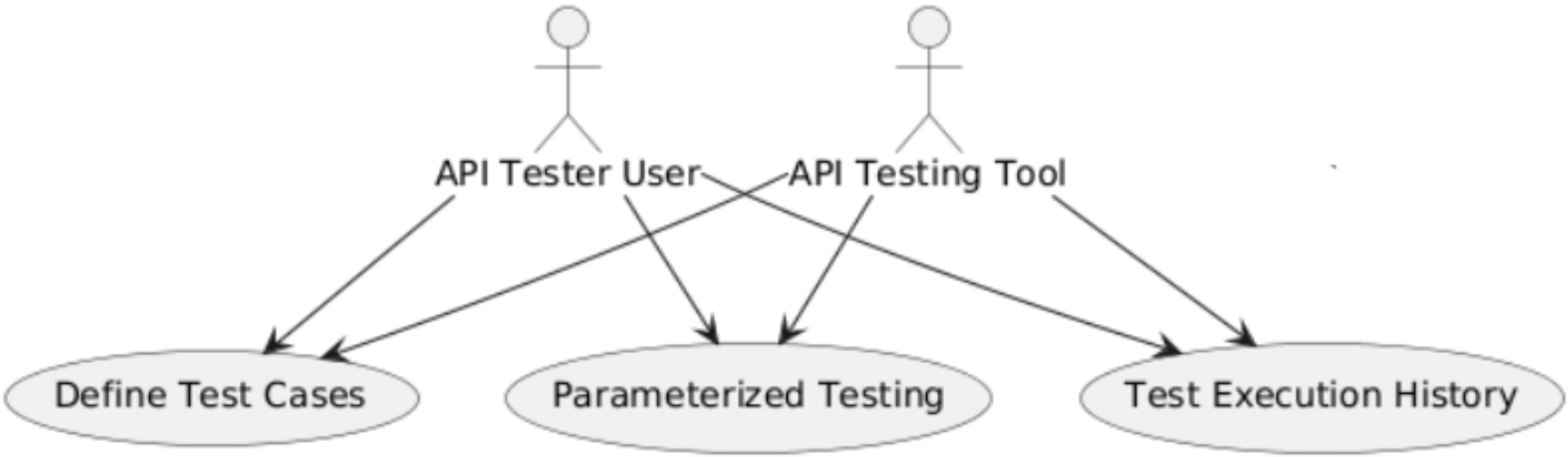
User module


API Tester User
API Testing Tool
API Server
Sign Up
Log In
Manage User Sessions
Store API Keys
Generate API Tokens

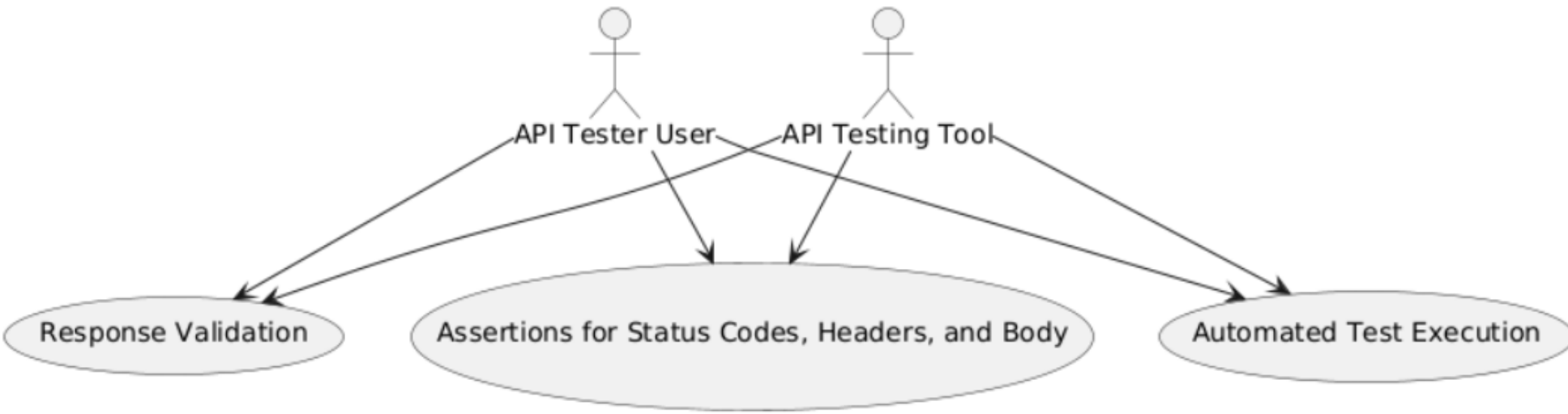
Request module



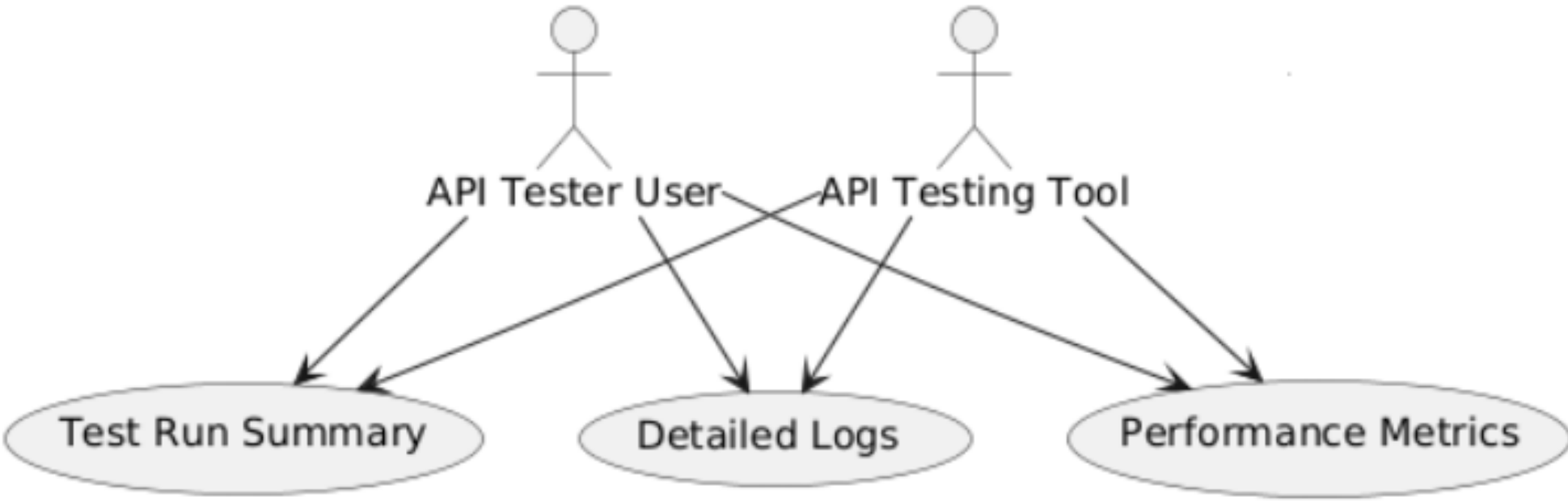
Testcase module



Assertion module



Report module



Algorithm and Procedure-Key Logic

The API Inspector follows a structured process for automated API testing:

- Executes API requests dynamically and captures responses.
- Automates validation using test cases and assertions.
- Handles errors by identifying API failures and providing debugging details.
- Generates reports summarizing test execution data for analysis.



User Authentication Module

Authenticates users and grants access based on credentials:

- User enters email and password.
- System verifies credentials in the database.
- If valid, generates an authentication token.
- If invalid, returns an error message.



Request Module

Handles API requests and response retrieval:

- User specifies API endpoint, HTTP method, headers, and payload.
- System sends the request and captures the response.
- Response data (status, headers, body) is stored for further validation.



Test Case Module

Defines, executes, and validates API test cases:

- User defines test cases with expected input and output.
- System executes the request and captures actual response.
- Actual response is compared with expected response.
- Test case is marked as **Pass** (if matched) or **Fail** (if mismatched).



Assertion Module

Ensures API responses meet expected conditions:

- User defines assertions (e.g., status code = 200, response contains key values).
- System checks if the response meets assertion criteria.
- If all assertions pass, the test is successful.
- If any assertion fails, the issue is highlighted.



Report Module

Generates test execution reports:

- Collects executed test case data.
- Formats test results with logs and failure reasons.



Coding Standards

API Inspector follows best practices for clean and maintainable code:

- Uses modular and reusable components for scalability.
- Follows consistent naming conventions (camelCase for variables, PascalCase for classes).
- Implements error handling with proper status codes and messages.
- Adheres to RESTful API design principles.
- Uses environment variables for sensitive configurations.



Setup Environment

Steps to set up the development environment:

- **Prerequisites:** Install Node.js , PostgreSQL and a package manager (npm/yarn).
- **Cloning the Repository:** `git clone https://github.com/username/api-inspector.git`
- **Installing Dependencies:** `cd api-inspector`
  `npm install`
- **Environment Configuration:** Create a `.env` file and set up variables.
- **Starting the Application:** `npm run dev` (for development mode)



Version Control

Managing API Inspector's source code using Git and GitLab:

- Uses Git for version control and GitLab for remote repository management.
- Follows a branching strategy (`main`, `develop`, and feature branches).
- Commit messages follow the convention: `[Module] Short Description`.
- Regular pull requests and code reviews ensure quality and collaboration.
- Automated CI/CD pipelines for deployment and testing.



Unit Testing - User Module

Front-end and Back-end Test Cases

Function	Test Case	Expected Result	Actual Result
User Authentication	Email: ziaraeez@gmail.com, Password: zia, CaptchaToken: invalid	Captcha verification failed	Captcha verification failed
User Authentication	Email: ziaraeez@gmail.com, Password: zia, Confirm Password: aaz, CaptchaToken: valid	Error message displayed	Error message displayed
Password Reset	Email: ziaraeez@gmail.com	Reset link sent via email	Reset link received



Unit Testing - Request Configuration

Function	Test Case	Expected Result
API Request	method: POST, url: `http://localhost:3000/api/login`, body: {}	Returns `400 Bad Request` due to missing fields
API Request	method: POST, url: (empty), body: {"username":"zia","password":"zia"}	Returns error due to missing URL
Header Configuration	Header: `Content-Type: application/json`	Request includes proper headers and is accepted by server



Module Interaction Tests

Integration testing ensures smooth interaction between different modules:

- Verifies data flow and communication between modules.
- Ensures correct API request execution and validation.
- Checks if assertion failures are correctly logged in reports.
- Confirms authentication mechanisms work with API requests.



Test Scenarios

Key integration test cases for API Inspector:

- **Authentication + Request Module:** - A logged-in user should be able to send API requests. - Unauthorized users should be denied access.
- **Request + Test Case Module:** - API responses should be matched against expected test case results.
- **Test Case + Assertion Module:** - Assertions should correctly validate API response structure and data.
- **Assertion + Report Module:** - Failed assertions should be recorded in reports with error details.



Testing Approach

Steps for integration testing:

1. Set up the test environment with required dependencies.
2. Use mock API responses for controlled testing.
3. Execute test cases covering multiple module interactions.
4. Validate logs, error handling, and response accuracy.
5. Generate reports and compare expected vs. actual outcomes.



Ensuring Correct Features

Functional testing verifies that each feature works as expected:

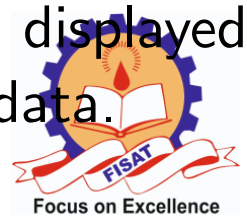
- Ensures API requests execute correctly with valid inputs.
- Confirms authentication, test case execution, and assertions function properly.
- Validates error handling for incorrect inputs and failed tests.
- Checks UI interactions (if applicable) for a smooth user experience.



Test Scenarios

Key functional test cases for API Inspector:

- **User Authentication:** - Users should successfully register and log in. - Invalid credentials should return an error.
- **API Request Execution:** - Requests should return correct responses based on input parameters. - Errors should be properly handled if the API is unreachable.
- **Test Case Execution:** - The system should correctly compare expected vs. actual responses. - Failed test cases should be flagged appropriately.
- **Assertions:** - Defined assertions should be correctly evaluated. - Mismatches should trigger failure notifications.
- **Report Generation:** - Test results should be recorded and displayed accurately. - Exported reports should match the displayed data.



Testing Approach

Steps for functional testing:

1. Identify key features and define test cases for each.
2. Execute test cases with valid and invalid inputs.
3. Verify that actual behavior matches expected behavior.
4. Log and analyze failures to identify bugs.
5. Retest after fixing issues to ensure stability.



Performance and Stress Tests

Load testing evaluates system behavior under different levels of demand:

- Measures API response time under normal and high traffic conditions.
- Identifies bottlenecks and ensures system scalability.
- Tests how the application handles a large volume of simultaneous requests.
- Determines system stability under extreme stress conditions.



Test Scenarios

Key load testing scenarios for API Inspector:

- **Performance Testing:** - Measure API response time with varying numbers of concurrent users. - Ensure the system maintains acceptable latency under peak load.
- **Stress Testing:** - Gradually increase request volume to identify the breaking point. - Observe how the system handles resource exhaustion (CPU, memory, database).
- **Scalability Testing:** - Evaluate system behavior with increased traffic and API requests. - Test auto-scaling capabilities (if applicable).
- **Failure Recovery:** - Simulate sudden traffic spikes and monitor system recovery. - Ensure proper handling of server crashes and timeouts.



Testing Approach

Steps for load testing:

1. Define performance benchmarks (acceptable response time, max requests per second).
2. Use load testing tools like JMeter, Locust, or k6 to simulate high traffic.
3. Gradually increase load and monitor system metrics (CPU, memory, response time).
4. Identify bottlenecks and optimize code, database queries, or server configurations.
5. Repeat tests after optimizations to validate improvements.



Bug Reports and Fixes

Reportee	Bug Description	Fix
Neha	User module registration failure due to missing validation checks	Fixed API call to properly handle response codes
Nandana	UI issue causing misalignment on smaller screens	Updated CSS for better responsiveness
Ziya	Database connection issue in production environment	Adjusted connection pooling and retry mechanism
Famitha	Incorrect error messages displayed for failed authentication	Standardized error handling messages
Ziya	API response delay for large payloads	Optimized server-side query and added pagination
Neha	Crash when uploading unsupported file types	Implemented file type validation before upload
Nandana	Headers/body not validated before stringifying	Added fallback to empty object before JSON.stringify
Ziya	AMQP connection remains open if error occurs after queue setup	Wrapped channel and connection closing in a `finally` block
Neha	No timeout while awaiting response from RabbitMQ	Added timeout handler to avoid hanging requests
Famitha	RabbitMQ message parsing may fail silently	Added try-catch around JSON.parse and error feedback



Cloud Hosting Details

Cloud Services Used: Amazon Web Services (AWS)

- **EC2 (Elastic Compute Cloud):**
  - Used to host and run the backend server (Node.js + Express).
  - Provided scalable and reliable virtual machines.
  - Enabled 24/7 availability and SSH access for deployment and debugging.
- **RDS (Relational Database Service):**
  - Hosted the PostgreSQL database for persistent storage.
  - Automated backups, scaling, and failover support.
  - Connected securely to EC2 instances via VPC for low-latency queries.



Status of work and contribution

- Famitha M T - Frontend request,report,testcase,assertion module
- Nandana S - Frontend of request ,report module,presentation
- Neha Amina Riyaz - Frontend user module,ci/cd ,docker,cloud
- Ziya K V - Backend -all module

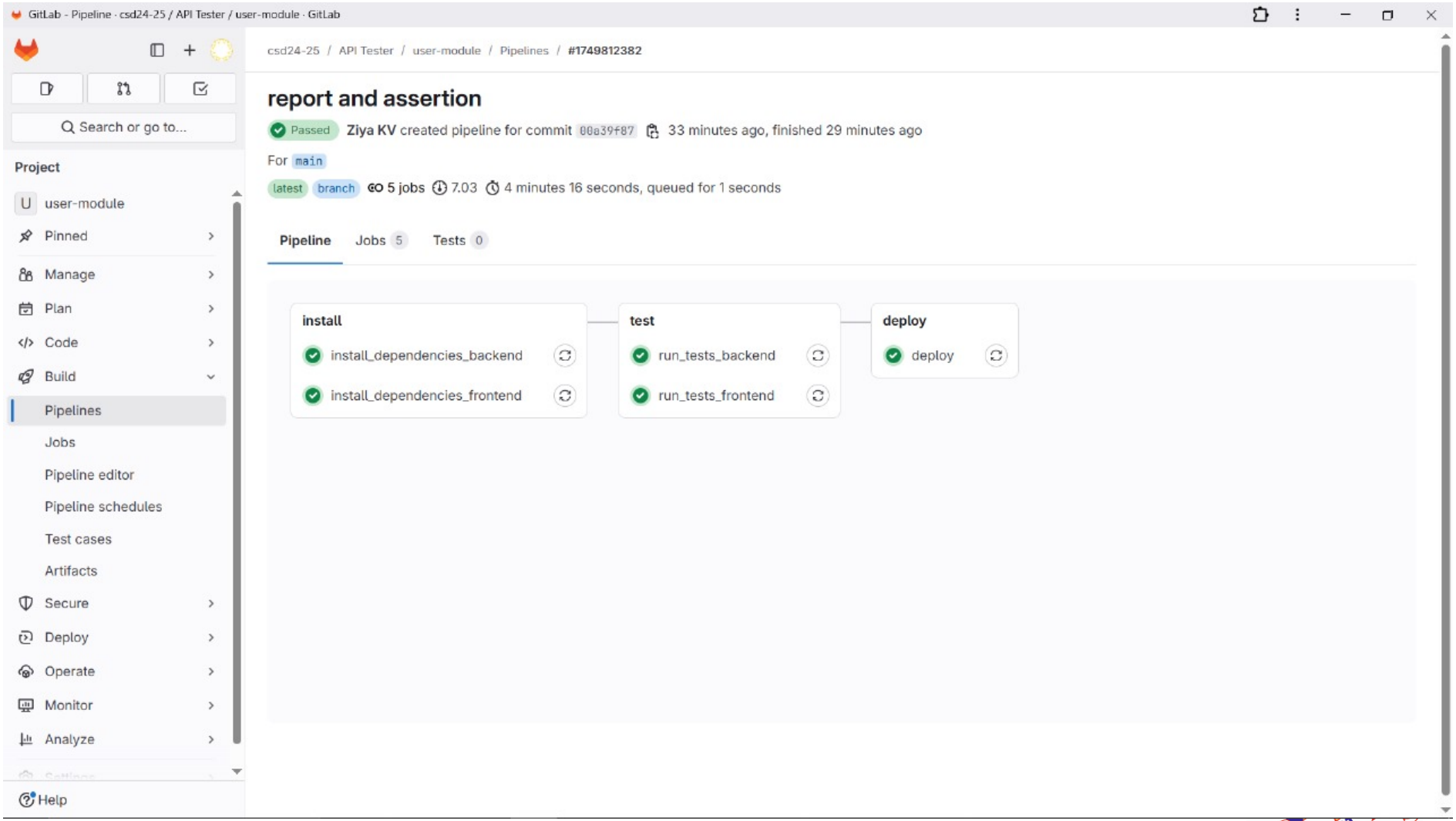


live or recorded model

YET TO ADD

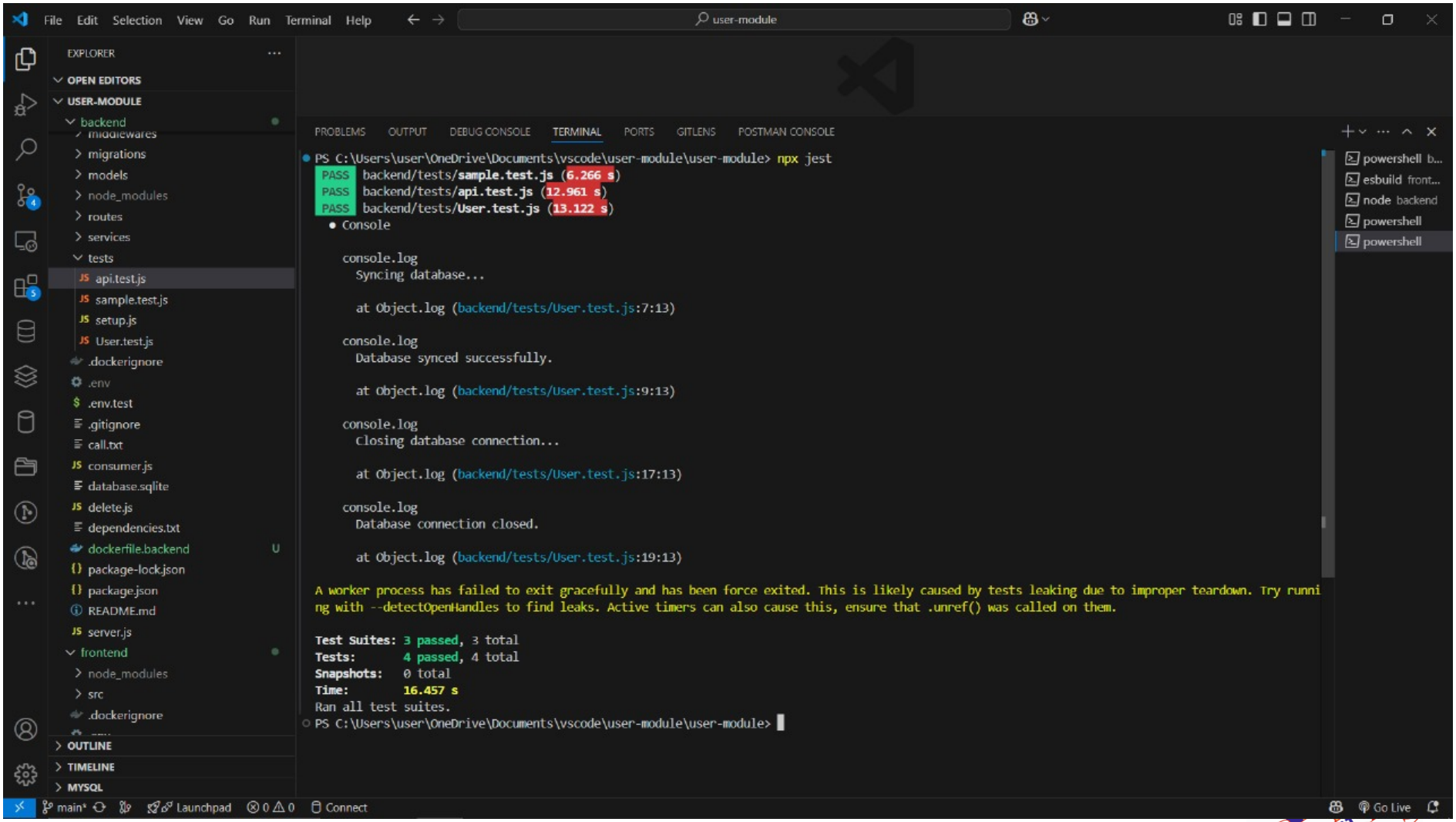


Supporting Visuals

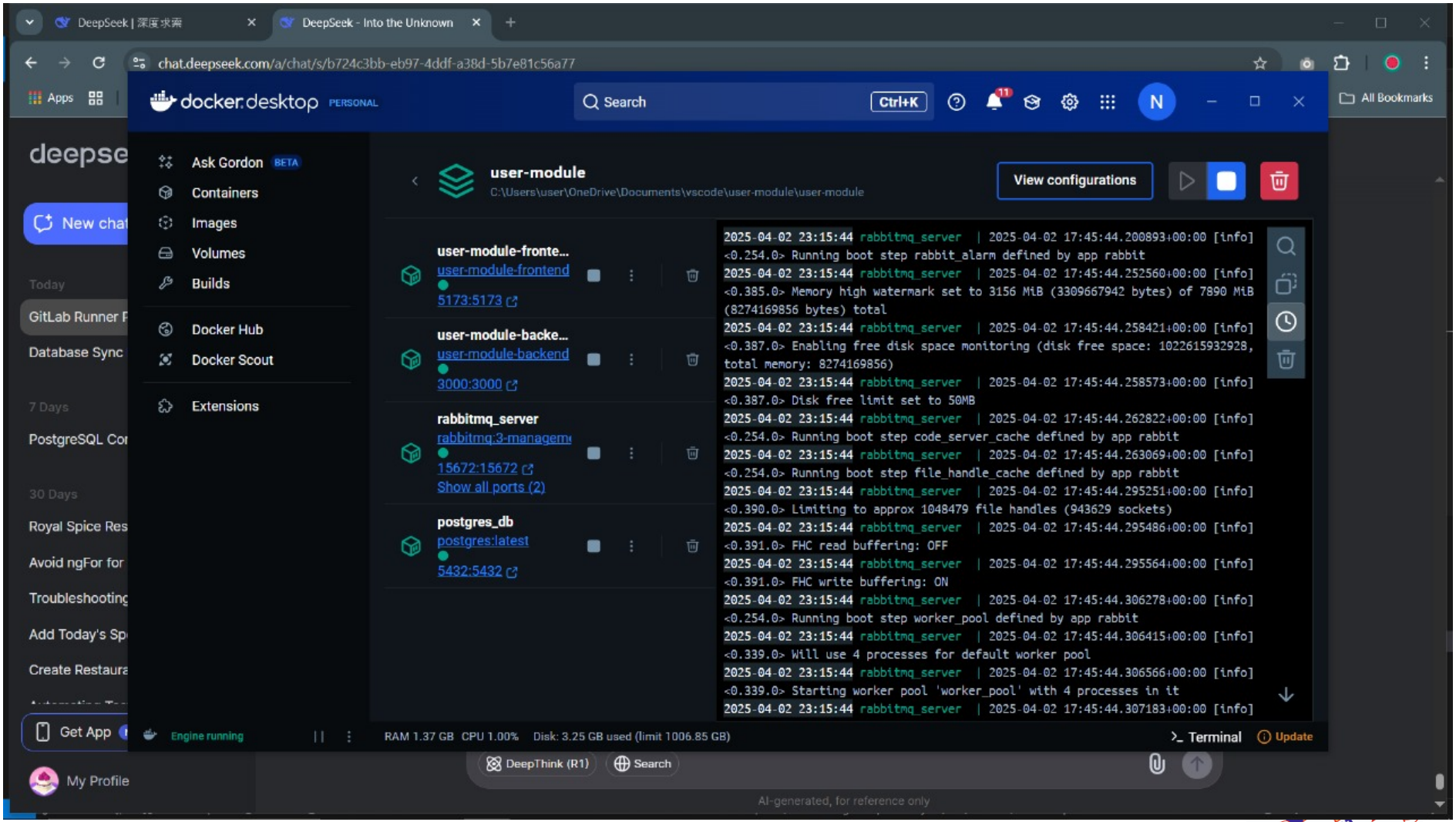


Focus on Excellence

Supporting Visuals



Supporting Visuals



Focus on Excellence

Cost Breakdown

No physical hardware was used in the development or testing of this project. All components were built and executed using existing local machines and open-source software tools. However, a minimal cost of Rs200 was incurred for using AWS cloud services.



Conclusion

- **Enhanced API Testing:** Provides a streamlined user experience for testing and debugging APIs.
- **Automated Validation:** Ensures APIs function correctly with automated test execution.
- **Performance Monitoring:** Tracks API response times and system behavior in real time.
- **Real-Time Request Analysis:** Allows users to inspect API requests and responses dynamically.
- **Improved Efficiency:** Reduces debugging time and accelerates the development process.
- **Reliability Assurance:** Helps ensure APIs are robust and function as expected.
- **Future Enhancements:** Potential improvements include:
  - AI-powered API analysis for intelligent debugging.
  - Enhanced security testing to detect vulnerabilities.
  - Better integrations with CI/CD pipelines for seamless deployment testing.



References

- SmartBear Software. "The Ultimate Guide to API Testing," 2021.
- Postman Team. "Postman API Platform Documentation," 2023.
- Swagger. "OpenAPI Specification Overview," 2022.
- Insomnia. "API Design and Collaboration Guide," 2023.
- Martin Fowler. "Continuous Integration and Deployment Best Practices," 2021.

